

AI HIRING BOT

Complete System Design & Implementation Blueprint

Powered by Ollama · llama3.1:8b · mistral:7b · nomic-embed-text

With pgvector Semantic Search — 100% Local, Zero Cloud LLM Cost

Version 3.0	Date June 2026	LLM Engine Ollama (local)	Vector DB pgvector
-----------------------	--------------------------	-------------------------------------	------------------------------

Table of Contents

Table of Contents.....	2
1. System Overview	4
1.1 Why Ollama + pgvector Together?	4
1.2 Hardware Fit — Your RTX 4050 6GB	4
1.3 Complete Model Assignment.....	4
1.4 Full Tech Stack.....	5
2. pgvector — Complete Guide	7
2.1 What Are Embeddings?	7
2.2 Installation & Setup.....	7
2.3 Distance Functions	7
2.4 Index Types for Fast Search.....	8
2.5 The Embedding Service	8
2.6 What pgvector Powers in This System	9
3. All 7 Layers — Complete Detail.....	11
4. pgvector-Powered Features	19
4.1 Semantic Resume Search	19
4.2 Find Similar Candidates.....	19
4.3 Duplicate Resume Detection	20
4.4 Historical Hire Matching.....	20
4.5 Auto Role Clustering.....	21
5. Database Schema with pgvector Columns.....	23
candidates	23
jobs	23
question_bank	24
screening_calls	24
score_reports.....	24
pgvector Indexes (run after table creation).....	25
6. Project Folder Structure	26
7. Dependencies & Environment.....	28
7.1 requirements.txt.....	28
7.2 docker-compose.yml	28
7.3 .env File.....	29
8. Implementation Roadmap	31
Phase 1 — Core Backend + pgvector Setup (Week 1–2)	31
Phase 2 — pgvector Features (Week 3).....	31

Phase 3 — Voice Engine (Week 4–5).....	31
Phase 4 — Scheduling + Notifications (Week 6).....	31
Phase 5 — HR Dashboard (Week 7–8).....	31
Phase 6 — Candidate Portal (Week 9).....	32
Phase 7 — Testing + Hardening (Week 10)	32
Phase 8 — Deployment (Week 11–12).....	32
9. Quick Start — From Zero to First Semantic Search.....	33

1. System Overview

The AI Hiring Bot is a fully automated, voice-driven recruitment screening system. It runs entirely on your local hardware using Ollama for all AI inference and pgvector inside PostgreSQL for semantic search and similarity matching. No cloud AI services are used — every piece of intelligence runs on your RTX 4050 laptop.

1.1 Why Ollama + pgvector Together?

The Core Philosophy

Ollama handles all language tasks: understanding resumes, generating questions, conducting interviews, writing reports

pgvector handles all similarity tasks: semantic search, candidate matching, duplicate detection, historical lookups

Together they give you a system that both understands language AND remembers meaning — just like a human HR expert

Everything stays on your machine: resumes, transcripts, embeddings, recordings — zero data leaves your network

Zero per-call AI cost: no Gemini, no OpenAI, no API bills — just your electricity

1.2 Hardware Fit — Your RTX 4050 6GB

Component	Memory Required	Fits in 6GB VRAM?	Notes
llama3.1:8b (analysis)	~5.5 GB VRAM	Yes — just fits	Used for Layers 2,3,4,6
mistral:7b (interview)	~4.1 GB VRAM	Yes — comfortable	Used for Layer 5 live call
nomic-embed-text (embed)	~0.3 GB VRAM	Yes — tiny	Used for Layer 1.5
Both llama3.1 + mistral	~9.6 GB VRAM	No — cannot co-run	Use Celery queue to alternate
nomic + mistral:7b	~4.4 GB VRAM	Yes — safe pair	Can run during call if needed

1.3 Complete Model Assignment

Layer	Task	Model	Speed	Why This Model
Layer 1	File parsing	PyMuPDF / python-docx	Instant	No LLM needed — pure file reading
Layer 1.5	Embedding generation	nomic-embed-text	~0.5s	Tiny, accurate, purpose-built for embeddings
Layer 2	Profile extraction	llama3.1:8b	~8s	Best structured JSON output, high accuracy

Layer 3	JD scoring	pgvector + llama3.1:8b	~2s+8s	Vector for speed, LLM for reasoning depth
Layer 4	Question generation	llama3.1:8b	~10s	Best instruction following, personalised Qs
Layer 5	Live interview	mistral:7b	~1.5s	Fastest response — critical for voice loop
Layer 6	Report generation	llama3.1:8b	~12s	Deep evaluation, runs after call ends

1.4 Full Tech Stack

Component	Technology	Purpose
LLM Engine	Ollama (local)	Runs all AI models on your GPU — no internet needed
Analysis Model	llama3.1:8b	Profile extraction, JD matching, question gen, report writing
Interview Model	mistral:7b	Real-time live call responses — speed optimised
Embedding Model	nomic-embed-text	Converts text to 768-dim vectors for semantic search
Vector Database	pgvector (PostgreSQL extension)	Stores and searches embeddings — semantic similarity
Primary Database	PostgreSQL 16	All candidate, job, call, and report data
Cache / State	Redis	Live call state, Celery task broker, session store
Task Queue	Celery + Redis	Background LLM jobs — prevents GPU VRAM conflicts
Resume Parsing	PyMuPDF + python-docx	Extract raw text from PDF and DOCX files
Voice Calls	Twilio Voice API	Outbound dialing, speech input gathering, webhooks
Text-to-Speech	ElevenLabs API	Natural HR voice — converts LLM text to audio
Speech-to-Text	Deepgram	Real-time transcription of candidate speech on call
Backend API	FastAPI (Python)	Core server, routing, auth, webhook handling
Frontend	React + Tailwind CSS	HR admin dashboard + Candidate portal
File Storage	Local disk / MinIO	Resume files, call recordings — no S3 needed
Notifications	Twilio SMS + SendGrid	Email and SMS to candidates and HR
Dev Tunnel	ngrok	Expose local webhooks to Twilio during development
Deployment	Docker + Nginx	Containerised production with GPU passthrough

2. pgvector — Complete Guide

pgvector is a PostgreSQL extension that adds a new column type called vector. You can store lists of floating-point numbers (embeddings) in these columns and run ultra-fast similarity searches across millions of rows using mathematical distance functions.

2.1 What Are Embeddings?

An embedding is what you get when an AI model reads a piece of text and converts it into a list of numbers that captures its meaning. Similar meanings produce similar numbers. This is the foundation of semantic search.

Embedding Intuition

'Python backend developer with FastAPI experience' → [0.23, 0.87, -0.45, 0.12, ...] 768 numbers

'Server-side engineer skilled in REST APIs and Python' → [0.21, 0.89, -0.43, 0.14, ...] 768 numbers

These two sentences mean the same thing — their vectors are very close together

'Marketing manager with social media expertise' → [-0.67, 0.12, 0.88, -0.34, ...] 768 numbers

This sentence means something very different — its vector is far away from the first two

Distance between vectors = semantic similarity — this is how pgvector search works

2.2 Installation & Setup

```
# — Step 1: Install pgvector extension —————
# On Ubuntu / Debian
sudo apt install postgresql-16-pgvector

# Or via Docker (recommended — already included in pgvector Docker image)
# In docker-compose.yml use image: pgvector/pgvector:pg16

# — Step 2: Enable extension in your database —————
-- Run this SQL once after connecting to your DB
CREATE EXTENSION IF NOT EXISTS vector;

# — Step 3: Install Python library —————
pip install pgvector

# — Step 4: Pull embedding model in Ollama —————
ollama pull nomic-embed-text

# — Step 5: Test embedding generation —————
ollama run nomic-embed-text 'Hello world'
# Should return a list of 768 floats
```

2.3 Distance Functions

pgvector supports three distance functions. Choose based on your use case:

Operator	Distance Type	SQL Syntax	Best For
<=>	Cosine distance	embedding <=> query_vec	Semantic similarity (our primary use)
<->	Euclidean (L2) distance	embedding <-> query_vec	Magnitude-sensitive comparisons
<#>	Inner product (negative)	embedding <#> query_vec	When vectors are normalised

We use cosine distance (<=>)

Cosine distance measures the ANGLE between two vectors, not their length

This means a short resume and a long resume about the same role will still match well

Range: 0.0 (identical meaning) to 2.0 (completely opposite meaning)

Similarity = 1 - cosine_distance → 1.0 = perfect match, 0.0 = no relation

nomic-embed-text produces normalised vectors, making cosine ideal

2.4 Index Types for Fast Search

Without an index, pgvector does exact nearest-neighbour search (slow at scale). With an index it uses approximate nearest-neighbour search (very fast, slightly less precise):

```
-- HNSW index — recommended for most use cases
-- Fast queries, good recall, works well up to millions of rows
CREATE INDEX ON candidates USING hnsw (resume_embedding vector_cosine_ops);
CREATE INDEX ON candidates USING hnsw (profile_embedding vector_cosine_ops);
CREATE INDEX ON jobs USING hnsw (jd_embedding vector_cosine_ops);
CREATE INDEX ON question_bank USING hnsw (question_embedding vector_cosine_ops);
CREATE INDEX ON score_reports USING hnsw (report_embedding vector_cosine_ops);

-- IVFFlat index — alternative, faster to build but lower recall
-- Use if you have 100k+ rows and HNSW build time is too slow
CREATE INDEX ON candidates USING ivfflat (resume_embedding vector_cosine_ops)
    WITH (lists = 100);
```

2.5 The Embedding Service

```
# backend/services/embedder.py
import ollama
from functools import lru_cache

MBED_MODEL = 'nomic-embed-text'
MBED_DIMS = 768

def embed_text(text: str) -> list[float]:
```



```

"""
Convert any text into a 768-dimensional embedding vector.
Uses nomic-embed-text via Ollama (local, no API key needed).
~0.3-0.5 seconds per call on RTX 4050.
"""

# Truncate to avoid token limit (nomic supports up to 8192 tokens)
text = text[:8000] if len(text) > 8000 else text
response = ollama.embeddings(
    model=EMBED_MODEL,
    prompt=text
)
return response['embedding'] # list of 768 floats

def embed_batch(texts: list[str]) -> list[list[float]]:
    """Embed multiple texts – used for bulk resume uploads."""
    return [embed_text(t) for t in texts]

def cosine_similarity(vec_a: list[float], vec_b: list[float]) -> float:
    """
    Compute cosine similarity between two vectors.
    Returns 0.0 (no similarity) to 1.0 (identical meaning).
    Used for in-memory comparisons without DB round-trip.
    """
    import math
    dot = sum(a*b for a,b in zip(vec_a, vec_b))
    mag_a = math.sqrt(sum(a*a for a in vec_a))
    mag_b = math.sqrt(sum(b*b for b in vec_b))
    if mag_a == 0 or mag_b == 0:
        return 0.0
    return dot / (mag_a * mag_b)

```

2.6 What pgvector Powers in This System

Feature	How pgvector is Used	SQL Distance Op
Semantic resume search	Embed HR's search query, find nearest resume vectors	<=>
JD matching (Layer 3)	Vector similarity score between resume and JD embeddings	<=>
Find similar candidates	Given a liked candidate, find others with closest embedding	<=>
Duplicate detection	If two resumes have similarity > 0.97, flag as duplicate	<=>
Historical hire matching	Find candidates similar to people hired in past roles	<=>
Question deduplication	Prevent near-identical questions in question bank	<=>
Auto role clustering	Group resumes by semantic similarity without a JD	<=>

3. All 7 Layers — Complete Detail

The system processes every candidate through 7 sequential layers. Layers 1-4 and 6 run as background Celery tasks. Layer 5 runs live during the phone call. Layer 1.5 runs in parallel with Layer 1.

Layer 1

Resume Ingestion — *PyMuPDF + python-docx — no LLM*

Opens the uploaded file and extracts all text. Handles PDF, DOCX, and plain text. Output is passed to Layer 1.5 and Layer 2 simultaneously.

```
# backend/services/resume_parser.py
import fitz # PyMuPDF
from docx import Document as DocxDoc
import os

def extract_resume_text(file_path: str) -> str:
    ext = os.path.splitext(file_path)[1].lower()
    if ext == '.pdf':
        doc = fitz.open(file_path)
        return '\n'.join(page.get_text() for page in doc)
    elif ext == '.docx':
        doc = DocxDoc(file_path)
        return '\n'.join(p.text for p in doc.paragraphs if p.text.strip())
    else:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()

def extract_jd_text(jd_input) -> str:
    """JD can be a file path or raw pasted text string."""
    if isinstance(jd_input, str) and os.path.exists(jd_input):
        return extract_resume_text(jd_input)
    return jd_input # already raw text
```

Layer 1.5

Embedding Generation (NEW — pgvector) — *nomic-embed-text via Ollama*

Converts resume text and JD text into 768-dimensional vectors and stores them in PostgreSQL via pgvector. This layer runs immediately after Layer 1 and enables all semantic features.

```
# backend/services/embedding_layer.py
from .embedder import embed_text
from ..database import get_db
from pgvector.sqlalchemy import Vector

async def generate_and_store_embeddings(
    candidate_id: str,
```

```

    resume_text: str,
    jd_text: str,
    db
) -> dict:
    """
    Layer 1.5: Generate embeddings for resume and JD.
    Stores vectors in pgvector columns.
    Runs concurrently with Layer 2 (no dependency between them).
    """
    # Generate embeddings
    resume_vec = embed_text(resume_text)          # ~0.4s on RTX 4050
    jd_vec      = embed_text(jd_text)              # ~0.4s on RTX 4050

    # Quick cosine similarity as a preview score (before full Layer 3)
    from .embedder import cosine_similarity
    quick_score = round(cosine_similarity(resume_vec, jd_vec) * 100, 1)

    # Store in database
    await db.execute("""
        UPDATE candidates
        SET resume_embedding = :rv,
            quick_match_score = :qs
        WHERE id = :cid
    """, {"rv": resume_vec, "qs": quick_score, "cid": candidate_id})

    await db.execute("""
        UPDATE jobs
        SET jd_embedding = :jv
        WHERE id = (SELECT jd_id FROM candidates WHERE id = :cid)
    """, {"jv": jd_vec, "cid": candidate_id})

    await db.commit()
    return {"resume_vector": resume_vec, "quick_score": quick_score}

```

Layer 2

Profile Extraction — *llama3.1:8b via Ollama*

Sends raw resume text to llama3.1:8b and extracts a rich structured JSON profile — skills, roles, education, projects, gaps, strengths, and red flags. Also generates a profile embedding stored in pgvector.

```

# backend/services/profile_extractor.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL
from .embedder import embed_text
import json

def extract_profile(resume_text: str) -> dict:
    prompt = f'''
    You are an expert HR analyst. Extract a complete structured JSON profile from this
    resume.
    Return ONLY valid JSON — no markdown, no explanation, no extra text.

```

JSON Schema:

```
{
  "name": string,
  "email": string,
  "phone": string,
  "skills": [list of all technical and soft skills],
  "experience_years": integer (total professional years),
  "roles": [{
    "title": str, "company": str,
    "duration": str, "responsibilities": [str]
  }],
  "education": [{{"degree": str, "institution": str, "year": str}},
  "projects": [{
    "name": str, "tech_stack": [str], "description": str
  }],
  "certifications": [str],
  "employment_gaps": [{{"period": str, "duration": str}},
  "strengths": [top 3 professional strengths],
  "red_flags": [list of concerns – gaps, short tenures, inconsistencies],
  "languages": [spoken and programming languages]
}
```

Resume Text:

```
{resume_text}
'''

profile = ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

# Also embed the profile JSON for richer semantic search
profile_summary = (
    f"{profile.get('name')} – {profile.get('experience_years')} years – "
    f"Skills: {' '.join(profile.get('skills', [])[:10])} – "
    f"Strengths: {' '.join(profile.get('strengths', []))}"
)
profile['profile_embedding'] = embed_text(profile_summary)
return profile
```

Layer 3

JD Matching & Scoring (pgvector UPGRADED) — *pgvector cosine + llama3.1:8b reasoning*

Two-stage scoring: pgvector gives a fast mathematical similarity score (40% weight), then llama3.1:8b does deep reasoning across 5 dimensions (60% weight). Combined score determines if candidate proceeds.

```
# backend/services/jd_matcher.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL

async def score_candidate_v2(
    candidate_id: str,
    profile: dict,
```

```

jd_text: str,
db
) -> dict:
    """
    Two-stage scoring:
    Stage 1: pgvector cosine similarity — fast, mathematical (40% weight)
    Stage 2: llama3.1:8b deep reasoning — slow, nuanced (60% weight)
    """

    # — Stage 1: Vector similarity (pgvector) —————
    vector_result = await db.execute("""
        SELECT
            ROUND((1 - (c.resume_embedding <=> j.jd_embedding)) * 100, 1)
            AS vector_similarity
        FROM candidates c
        JOIN jobs j ON c.jd_id = j.id
        WHERE c.id = :cid
    """, {"cid": candidate_id})
    vector_score = vector_result.scalar() or 0.0

    # — Stage 2: LLM deep scoring (llama3.1:8b) —————
    prompt = f'''
You are a senior HR recruiter. Score this candidate against the job description.
Return ONLY valid JSON:
{{
    "skills_score": 0-100,
    "experience_score": 0-100,
    "education_score": 0-100,
    "project_score": 0-100,
    "red_flag_penalty": 0-100 (100 = no penalty, 0 = severe red flags),
    "matched_skills": [list],
    "missing_skills": [list],
    "reasoning": "2-3 sentence match summary"
}}
Candidate Profile: {profile}
Job Description: {jd_text}
'''

    llm_scores = ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

    # — Combine scores —————
    llm_overall = (
        llm_scores['skills_score']      * 0.30 +
        llm_scores['experience_score']  * 0.25 +
        llm_scores['education_score']   * 0.15 +
        llm_scores['project_score']     * 0.20 +
        llm_scores['red_flag_penalty']  * 0.10
    )
    final_score = round((vector_score * 0.40) + (llm_overall * 0.60), 1)

    return {
        'vector_score': vector_score,
        'llm_score': round(llm_overall, 1),
    }

```

```

        'final_score': final_score,
        'decision': get_decision(final_score),
        **llm_scores
    }

def get_decision(score: float) -> str:
    if score < 40: return 'reject'
    if score < 60: return 'review'
    return 'proceed'

```

Layer 4

Question Generation — *llama3.1:8b via Ollama*

Generates 8-12 personalised screening questions based on the specific candidate profile and JD. Uses the candidate's actual skills, gaps, and employment history to craft targeted questions.

```

# backend/services/question_gen.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL
from .embedder import embed_text

async def generate_questions(
    profile: dict,
    jd_text: str,
    count: int = 10,
    db = None
) -> list[dict]:
    """
    Generate personalised questions.
    Checks pgvector question bank first to avoid duplicates.
    """
    prompt = f'''
You are an expert technical interviewer. Generate {count} screening questions
for this specific candidate applying for this specific role.
Return ONLY a valid JSON array:
[{{
    "id": 1,
    "type": "warmup|technical|behavioral|resume_probe|situational|closing",
    "question": "Full question text",
    "ideal_answer_points": ["key points to listen for in a good answer"],
    "follow_up": "Follow-up question if answer is vague or too brief"
}}]

Rules:
- First 1-2 must be warmup (easy, conversational)
- Probe every employment gap and short tenure you see
- Ask about specific skills listed in JD that candidate claims to know
- Last question must be closing (invite candidate questions)
- Keep questions conversational – this is a phone call, not a written test

Candidate Profile: {profile}

```

```

Job Description: {jd_text}
'''

questions = ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

# pgvector deduplication: check question bank for near-duplicates
if db:
    questions = await deduplicate_questions(questions, db)

return questions

async def deduplicate_questions(questions: list, db) -> list:
    """Remove questions too similar to existing question bank entries."""
    unique = []
    for q in questions:
        q_vec = embed_text(q['question'])
        result = await db.execute("""
            SELECT 1 - (question_embedding <=> :vec) AS similarity
            FROM question_bank
            ORDER BY question_embedding <=> :vec
            LIMIT 1
        """, {"vec": q_vec})
        row = result.fetchone()
        # Only add if no existing question is > 90% similar
        if not row or row.similarity < 0.90:
            unique.append(q)
    return unique

```

Layer 5

Voice Interview Engine — *mistral:7b (live) + Twilio + Deepgram + ElevenLabs*

The live phone call loop. Twilio dials the candidate, Deepgram transcribes their speech, mistral:7b generates the next response in ~1.5 seconds, ElevenLabs speaks it aloud. Runs in real time for 12-15 minutes.

```

# backend/services/interview_engine.py
from .ollama_client import ollama_chat, INTERVIEW_MODEL

def generate_next_response(call_state: dict) -> str:
    """
    Core Layer 5 function — called on every candidate speech turn.
    Uses mistral:7b for ~1.5s response time on RTX 4050.
    """
    transcript_text = '\n'.join(
        f"{t['speaker'].upper(): t['text']}"
        for t in call_state['transcript'][-10:] # Last 10 turns for context
    )
    current_q_idx = call_state['current_q_index']
    total_qs = len(call_state['questions'])

    if current_q_idx < total_qs:
        current_q = call_state['questions'][current_q_idx]

```



```

        next_q = call_state['questions'][current_q_idx+1] if current_q_idx+1 <
total_qs else None
    else:
        return generate_closing(call_state)

    prompt = f'''
You are Aria, a professional AI HR assistant on a phone screening call.
Be warm, concise, and professional. Max 3 sentences per response.
Phase: {call_state['phase']} | Question {current_q_idx+1} of {total_qs}

Candidate: {call_state['candidate_name']} | Role: {call_state['role']}
Current question: {current_q['question']}
Follow-up available: {current_q.get('follow_up', 'none')}

Recent conversation:
{transcript_text}

Instructions:
- Briefly acknowledge the last answer (1 sentence max)
- If answer was too vague or short, use the follow-up question instead of moving on
- Otherwise transition smoothly to the next question
- Never repeat a question — check the transcript
- Sound human and natural — this is a real phone call
- Respond with ONLY what Aria says next. No labels, no quotes.
'''

    return ollama_chat(prompt, model=INTERVIEW_MODEL, expect_json=False)

def generate_closing(call_state: dict) -> str:
    return (
        f"Thank you so much for your time, {call_state['candidate_name']}. "
        "That's all the questions I have. Do you have anything you'd like to ask "
        "about the role or the company?"
    )

```

Layer 6

Report Generation + Report Embedding — *llama3.1:8b via Ollama + nomic-embed-text*

Reads the full transcript and generates a detailed score card with 6 dimensions, AI recommendation with reasoning, strengths, red flags, and next-round questions. Also embeds the report for historical similarity search.

```

# backend/services/report_gen.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL
from .embedder import embed_text

def generate_report(call_state: dict, profile: dict) -> dict:
    transcript_text = '\n'.join(
        f"{t['speaker'].upper(): {t['text']}"
        for t in call_state['transcript']
    )

```

```
prompt = f'''
You are a senior HR evaluator. Analyse this screening call and produce a
detailed candidate evaluation report. Return ONLY valid JSON:
{{
  "overall_score": 0-100,
  "skills_score": 0-100,
  "experience_score": 0-100,
  "communication_score": 0-100,
  "culture_fit_score": 0-100,
  "confidence_score": 0-100,
  "recommendation": "HIRE|SHORTLIST|HOLD|REJECT",
  "reasoning": "3-4 sentence detailed justification for the recommendation",
  "strengths": ["list of specific strengths observed in answers"],
  "red_flags": ["list of specific concerns from the call"],
  "next_round_questions": ["targeted questions for round 2 if they proceed"]
}}

Candidate: {call_state['candidate_name']} | Role: {call_state['role']}
Pre-call JD match score: {call_state['match_score']}%
Call duration: {call_state.get('duration_seconds', 0)} seconds

Full interview transcript:
{transcript_text}
'''

report = ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

# Embed report summary for historical similarity search (pgvector)
report_summary = (
    f"{call_state['candidate_name']} - {call_state['role']} - "
    f"Score: {report['overall_score']} - {report['recommendation']} - "
    f"Strengths: {'', '.join(report.get('strengths', []))} - "
    f"Flags: {'', '.join(report.get('red_flags', []))}"
)
report['report_embedding'] = embed_text(report_summary)
return report
```

4. pgvector-Powered Features

These are the new features that pgvector unlocks on top of the base hiring bot. All are implemented as new API endpoints in the HR dashboard.

4.1 Semantic Resume Search

HR types a natural language query into the search bar. The query is embedded and compared against all resume vectors. Returns ranked candidates by meaning, not keywords.

```
# backend/routers/hr.py
@router.get('/candidates/semantic-search')
async def semantic_search(
    query: str,
    limit: int = 10,
    min_similarity: float = 0.60,
    db = Depends(get_db)
):
    """
    Example queries that work:
    - 'experienced Python backend developer'
    - 'someone who has built microservices at scale'
    - 'full stack engineer with React and Node'
    - 'data engineer with Spark and Kafka experience'
    """
    query_vec = embed_text(query)
    results = await db.execute("""
        SELECT
            c.id, c.name, c.email, c.status,
            j.title AS role,
            c.match_score,
            ROUND((1 - (c.resume_embedding <=> :qv))::numeric * 100, 1)
            AS semantic_similarity
        FROM candidates c
        JOIN jobs j ON c.jd_id = j.id
        WHERE (1 - (c.resume_embedding <=> :qv)) >= :min_sim
        ORDER BY c.resume_embedding <=> :qv
        LIMIT :limit
    """, {"qv": query_vec, "min_sim": min_similarity, "limit": limit})
    return results.mappings().all()
```

4.2 Find Similar Candidates

HR clicks any candidate and sees the top 5 most similar candidates in the database. Useful for building a shortlist or finding backup options.

```
@router.get('/candidates/{candidate_id}/similar')
async def find_similar_candidates(
```

```

candidate_id: str,
limit: int = 5,
db = Depends(get_db)
):
    results = await db.execute("""
        SELECT
            c2.id, c2.name, c2.status, c2.match_score,
            ROUND((1 - (c1.resume_embedding <=> c2.resume_embedding))
                ::numeric * 100, 1) AS similarity_pct
        FROM candidates c1, candidates c2
        WHERE c1.id = :cid
            AND c2.id != :cid
        ORDER BY c1.resume_embedding <=> c2.resume_embedding
        LIMIT :limit
    """, {"cid": candidate_id, "limit": limit})
    return results.mappings().all()

```

4.3 Duplicate Resume Detection

Automatically flags when the same candidate submits twice under different details. Runs on every new upload before processing begins.

```

async def check_duplicate(resume_embedding: list[float], db) -> dict | None:
    """
    Returns existing candidate if similarity > 97%.
    97% threshold catches reformatted resumes, name/email changes.
    """
    result = await db.execute("""
        SELECT id, name, email,
            ROUND((1 - (resume_embedding <=> :vec))::numeric * 100, 1)
            AS similarity
        FROM candidates
        WHERE (1 - (resume_embedding <=> :vec)) > 0.97
        ORDER BY resume_embedding <=> :vec
        LIMIT 1
    """, {"vec": resume_embedding})
    row = result.mappings().fetchone()
    return dict(row) if row else None

```

4.4 Historical Hire Matching

Find candidates similar to people who were successfully hired in the past. HR marks past successful hires, and the system learns which profiles to prioritise.

```

@router.get('/candidates/similar-to-hires')
async def find_candidates_like_past_hires(
    role_id: str,
    limit: int = 10,
    db = Depends(get_db)
)

```

```

):
    """
    Find new applicants whose resumes are similar to
    candidates who were previously HIRED for this role.
    """
    results = await db.execute("""
        SELECT
            new_c.id, new_c.name, new_c.match_score,
            ROUND(AVG(1 - (new_c.resume_embedding <=> hired.resume_embedding))
                * 100, 1) AS avg_similarity_to_hires
        FROM candidates new_c
        CROSS JOIN (
            SELECT c.resume_embedding
            FROM candidates c
            JOIN score_reports sr ON sr.call_id IN (
                SELECT id FROM screening_calls WHERE candidate_id = c.id
            )
            WHERE sr.hr_override = 'HIRE'
                AND c.jd_id = :role_id
        ) AS hired
        WHERE new_c.status IN ('pending', 'scheduled')
            AND new_c.jd_id = :role_id
        GROUP BY new_c.id, new_c.name, new_c.match_score
        ORDER BY avg_similarity_to_hires DESC
        LIMIT :limit
    """, {"role_id": role_id, "limit": limit})
    return results.mappings().all()

```

4.5 Auto Role Clustering

Groups all candidates by semantic similarity without requiring a JD. Useful when HR has a pool of unsorted applications.

```

# Simple cluster preview — returns candidates grouped by similarity
@router.get('/candidates/cluster')
async def cluster_candidates(
    threshold: float = 0.75,
    db = Depends(get_db)
):
    """
    Returns candidate pairs with high resume similarity.
    Front-end can use this to draw cluster groups visually.
    """
    results = await db.execute("""
        SELECT
            c1.id AS id_a, c1.name AS name_a,
            c2.id AS id_b, c2.name AS name_b,
            ROUND((1 - (c1.resume_embedding <=> c2.resume_embedding))
                ::numeric * 100, 1) AS similarity
        FROM candidates c1
        JOIN candidates c2 ON c2.id > c1.id
    """)

```

```
WHERE (1 - (c1.resume_embedding <=> c2.resume_embedding)) >= :thresh
ORDER BY similarity DESC
""", {"thresh": threshold})
return results.mappings().all()
```

5. Database Schema with pgvector Columns

All vector columns use the pgvector vector(768) type — matching the output dimensions of nomic-embed-text. HNSW indexes are created on all embedding columns for fast approximate nearest-neighbour search.

candidates

Column	Type	Description
id	UUID PK	Unique candidate ID
name	VARCHAR	Full name
email	VARCHAR UNIQUE	Email address
phone	VARCHAR	Phone in E.164 format
resume_url	TEXT	Local path to resume file
jd_id	UUID FK	Linked job description
profile_json	JSONB	Layer 2 extracted profile
resume_embedding	vector(768) ★	Layer 1.5: full resume text embedding
profile_embedding	vector(768) ★	Layer 2: profile summary embedding
match_score	INTEGER	Layer 3 combined final score 0-100
vector_score	FLOAT	Layer 3 pgvector component score
llm_score	FLOAT	Layer 3 LLM reasoning component score
match_details	JSONB	Per-dimension scores and matched/missing skills
status	ENUM	pending/scheduled/in_call/completed/rejected
consent_given	BOOLEAN	AI call consent confirmed
consent_at	TIMESTAMP	When consent was given
created_at	TIMESTAMP	Application submitted timestamp

jobs

Column	Type	Description
id	UUID PK	Unique job ID
title	VARCHAR	Job title
department	VARCHAR	Department name
jd_text	TEXT	Full job description text
jd_embedding	vector(768) ★	Layer 1.5: JD text embedding for matching
required_skills	JSONB	Parsed required skills
min_experience	INTEGER	Minimum years required

auto_reject_threshold	INTEGER	Min match score (default: 40)
question_count	INTEGER	Questions to generate (default: 10)
created_at	TIMESTAMP	Job created timestamp

question_bank

Column	Type	Description
id	UUID PK	Question ID
jd_id	UUID FK	Linked job (null = universal question)
question_text	TEXT	The question
question_embedding	vector(768) ★	Used for deduplication — Layer 4
type	ENUM	warmup/technical/behavioral/situational/closing
difficulty	ENUM	easy/medium/hard
pinned	BOOLEAN	Always include for this role if true
created_at	TIMESTAMP	When question was added

screening_calls

Column	Type	Description
id	UUID PK	Call record ID
candidate_id	UUID FK	Linked candidate
twilio_call_sid	VARCHAR	Twilio call SID
scheduled_date	DATE	Date set by HR
scheduled_time	TIME	Time chosen by candidate
call_started_at	TIMESTAMP	Actual call start
call_ended_at	TIMESTAMP	Actual call end
duration_seconds	INTEGER	Call length
recording_url	TEXT	Twilio recording URL
transcript	JSONB	[{speaker, text, timestamp}]
retry_count	INTEGER	Retry attempts made
interview_model	VARCHAR	Ollama model used (mistral:7b)
status	ENUM	pending/dialing/in_progress/completed/no_answer/failed

score_reports

Column	Type	Description
id	UUID PK	Report ID
call_id	UUID FK	Linked screening call

overall_score	INTEGER	0-100 overall score
skills_score	INTEGER	Technical skills score
experience_score	INTEGER	Experience relevance score
communication_score	INTEGER	Communication clarity score
culture_fit_score	INTEGER	Culture fit score
confidence_score	INTEGER	Confidence and engagement score
ai_recommendation	ENUM	HIRE/SHORTLIST/HOLD/REJECT
ai_reasoning	TEXT	LLM justification for recommendation
strengths	JSONB	Observed strengths list
red_flags	JSONB	Concerns from call list
next_round_questions	JSONB	Suggested round 2 questions
report_embedding	vector(768) ★	Layer 6: report summary embedding for history search
report_model	VARCHAR	llama3.1:8b
hr_override	ENUM	HR manual override (nullable)
hr_notes	TEXT	HR additional comments

pgvector Indexes (run after table creation)

```
-- HNSW indexes for all vector columns
-- Build once, query in microseconds

CREATE INDEX idx_candidates_resume_vec
  ON candidates USING hnsw (resume_embedding vector_cosine_ops);

CREATE INDEX idx_candidates_profile_vec
  ON candidates USING hnsw (profile_embedding vector_cosine_ops);

CREATE INDEX idx_jobs_jd_vec
  ON jobs USING hnsw (jd_embedding vector_cosine_ops);

CREATE INDEX idx_question_bank_vec
  ON question_bank USING hnsw (question_embedding vector_cosine_ops);

CREATE INDEX idx_score_reports_vec
  ON score_reports USING hnsw (report_embedding vector_cosine_ops);

-- ★ = pgvector column (vector(768) type)
-- nomic-embed-text outputs exactly 768 dimensions
```

6. Project Folder Structure

```

hiring-bot/
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── database.py           # PostgreSQL + pgvector setup
│   ├── redis_client.py
│   ├── celery_app.py
│   ├── tasks.py
│   └── routers/
│       └── hr.py           # Includes semantic search, similar, cluster
├── endpoints
│   ├── candidate.py
│   ├── voice.py
│   └── reports.py
│   └── services/
│       ├── ollama_client.py # Shared Ollama wrapper
│       ├── embedder.py      # nomic-embed-text + cosine_similarity
│       ├── resume_parser.py # Layer 1
│       ├── embedding_layer.py # Layer 1.5 - pgvector storage
│       ├── profile_extractor.py # Layer 2
│       ├── jd_matcher.py    # Layer 3 - pgvector + llama3.1:8b
│       ├── question_gen.py  # Layer 4 - with deduplication
│       ├── interview_engine.py # Layer 5 - mistral:7b
│       └── report_gen.py    # Layer 6 - with report embedding
│   └── voice/
│       ├── twilio_client.py
│       ├── tts.py
│       └── call_state.py
│   └── notifications/
│       ├── email.py
│       └── sms.py
│   └── models/           # SQLAlchemy models with Vector columns
│       ├── candidate.py # resume_embedding = mapped_column(Vector(768))
│       ├── job.py       # jd_embedding = mapped_column(Vector(768))
│       ├── call.py
│       ├── report.py    # report_embedding = mapped_column(Vector(768))
│       └── question_bank.py # question_embedding =
│   mapped_column(Vector(768))
│   └── schemas/
│       ├── candidate.py
│       └── search.py    # SemanticSearchRequest,
│   SimilarCandidatesResponse
│       └── report.py
├── frontend/
│   ├── hr-dashboard/
│   │   └── src/
│   │       ├── pages/
│   │       └── Overview.jsx

```

```
|
|
|   |— Upload.jsx
|   |— Candidates.jsx      # Semantic search bar added
|   |— Schedule.jsx
|   |— InterviewLogs.jsx
|   |— SimilarCandidates.jsx # New pgvector feature
|   |— Settings.jsx
|   |— components/
|   |— candidate-portal/
|       |— src/pages/
|           |— Apply.jsx
|           |— Status.jsx
|           |— PickSlot.jsx
|           |— PreCall.jsx
|           |— PostCall.jsx
|
|— alembic/                                # DB migrations (includes pgvector columns)
|— audio_cache/
|— uploads/
|— docker-compose.yml                    # Uses pgvector/pgvector:pg16 image
|— requirements.txt
|— .env.example
|— README.md
```

7. Dependencies & Environment

7.1 requirements.txt

```
# Core framework
fastapi==0.111.0
uvicorn==0.29.0
python-multipart==0.0.9

# Database + pgvector
sqlalchemy==2.0.30
alembic==1.13.1
asyncpg==0.29.0
pgvector==0.3.2          # pgvector Python client + SQLAlchemy integration
psycopg2-binary==2.9.9  # Sync driver for migrations

# Cache + Queue
redis==5.0.4
celery==5.4.0
flower==2.0.1

# Ollama (all 3 models - analysis, interview, embedding)
ollama==0.2.1

# Resume parsing
PyMuPDF==1.24.3
python-docx==1.1.2

# Voice
twilio==9.0.5
deepgram-sdk==3.2.7
elevenlabs==1.1.3

# Notifications
sendgrid==6.11.0

# Scheduling + Auth
apscheduler==3.10.4
pydantic==2.7.1
pydantic-settings==2.3.0
python-jose==3.3.0
passlib==1.7.4
httpx==0.27.0
```

7.2 docker-compose.yml

```
version: '3.9'
services:

    # PostgreSQL with pgvector extension built in
```

```
db:
  image: pgvector/pgvector:pg16 # ← important: use this image, not plain
postgres
  environment:
    POSTGRES_DB: hiringbot
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports:
    - '5432:5432'

redis:
  image: redis:7-alpine
  ports:
    - '6379:6379'

# Ollama with GPU passthrough
ollama:
  image: ollama/ollama
  runtime: nvidia
  environment:
    - NVIDIA_VISIBLE_DEVICES=all
  volumes:
    - ollama_models:/root/.ollama
  ports:
    - '11434:11434'

backend:
  build: .
  env_file: .env
  depends_on: [db, redis, ollama]
  ports:
    - '8000:8000'

volumes:
  pgdata:
  ollama_models:
```

7.3 .env File

```
# — Ollama —————
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_ANALYSIS_MODEL=llama3.1:8b
OLLAMA_INTERVIEW_MODEL=mistral:7b
OLLAMA_EMBED_MODEL=nomic-embed-text
OLLAMA_EMBED_DIMS=768

# — Database + pgvector —————
DATABASE_URL=postgresql+asyncpg://user:password@localhost/hiringbot
REDIS_URL=redis://localhost:6379
```

```
# — Twilio —————
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE_NUMBER=+1XXXXXXXXXX
```

```
# — ElevenLabs —————
ELEVENLABS_API_KEY=your_key
ELEVENLABS_VOICE_ID=your_voice_id
```

```
# — Deepgram —————
DEEPGRAM_API_KEY=your_key
```

```
# — SendGrid —————
SENDGRID_API_KEY=your_key
FROM_EMAIL=hr@yourcompany.com
```

```
# — App —————
SECRET_KEY=your_jwt_secret
WEBHOOK_BASE_URL=https://your-ngrok-url.ngrok.io
UPLOAD_DIR=./uploads
AUDIO_CACHE_DIR=./audio_cache
AUTO_REJECT_THRESHOLD=40
VECTOR_SIMILARITY_WEIGHT=0.40
LLM_SCORE_WEIGHT=0.60
DUPLICATE_THRESHOLD=0.97
QUESTION_DEDUP_THRESHOLD=0.90
```

8. Implementation Roadmap

Phase 1 — Core Backend + pgvector Setup (Week 1–2)

- Install Ollama, pull llama3.1:8b, mistral:7b, nomic-embed-text
- Set up FastAPI, PostgreSQL with pgvector/pgvector:pg16 Docker image
- Run CREATE EXTENSION vector — verify with SELECT '[1,2,3]':vector
- Set up Redis, Celery with analysis and notification queues
- Implement Layer 1: resume ingestion (PDF + DOCX)
- Implement Layer 1.5: embedding generation + pgvector storage
- Implement Layer 2: profile extraction with llama3.1:8b
- Implement Layer 3: combined pgvector + LLM scoring
- Test end-to-end: upload resume → get match score in ~20 seconds

Phase 2 — pgvector Features (Week 3)

- Implement semantic search endpoint
- Implement find-similar-candidates endpoint
- Implement duplicate detection — runs on every new upload automatically
- Implement Layer 4 with question deduplication via pgvector
- Create HNSW indexes on all vector columns
- Test semantic search with diverse queries

Phase 3 — Voice Engine (Week 4–5)

- Set up Twilio, buy phone number, configure webhooks via ngrok
- Build voice webhook handlers (/voice/start, /voice/respond, /voice/status)
- Integrate ElevenLabs TTS with audio caching
- Implement Redis call state manager
- Wire mistral:7b into live interview engine
- Implement Layer 6 report generation with report embedding
- Test full end-to-end: upload → call → report → pgvector search

Phase 4 — Scheduling + Notifications (Week 6)

- APScheduler for timed call triggering
- SendGrid email + Twilio SMS integration
- Candidate time slot selection API
- .ics calendar invite generation
- No-answer retry logic (3 retries, 30 min apart)

Phase 5 — HR Dashboard (Week 7–8)

- Overview dashboard, upload module, candidate list with semantic search bar
- Schedule module, interview logs with transcript viewer
- Similar candidates panel (powered by pgvector)
- Cluster view — visual grouping of candidate pool
- Historical hire matching panel
- Question bank editor and settings page

Phase 6 — Candidate Portal (Week 9)

- 4-step apply flow, status timeline, time slot picker
- Pre-call guidance, AI consent screen, post-call confirmation

Phase 7 — Testing + Hardening (Week 10)

- End-to-end call tests with real phone numbers
- pgvector accuracy audit: verify semantic search returns relevant results
- GPU conflict stress test: simulate upload during active call
- Prompt tuning: improve JSON consistency from llama3.1:8b
- Security: JWT auth, rate limiting, input sanitisation

Phase 8 — Deployment (Week 11–12)

- Docker Compose with pgvector:pg16 image and Ollama GPU container
- Nginx reverse proxy with SSL (Let's Encrypt)
- Twilio production webhooks (replace ngrok)
- Monitoring: Sentry, Flower for Celery, nvidia-smi health check
- Final UAT and go-live

9. Quick Start — From Zero to First Semantic Search

1. Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
2. Pull all 3 models: `ollama pull llama3.1:8b && ollama pull mistral:7b && ollama pull nomic-embed-text`
3. Start Ollama: `ollama serve` (keep terminal open)
4. Start PostgreSQL with pgvector: `docker run -e POSTGRES_PASSWORD=pass -p 5432:5432 pgvector/pgvector:pg16`
5. Connect to DB and enable extension: `CREATE EXTENSION vector;`
6. Start Redis: `docker run -p 6379:6379 redis:7-alpine`
7. Install Python deps: `pip install -r requirements.txt`
8. Copy `.env.example` to `.env` — fill in Twilio, ElevenLabs, SendGrid keys
9. Run DB migrations: `alembic upgrade head` (creates all tables + vector columns + HNSW indexes)
10. Start FastAPI: `uvicorn backend.main:app --reload --port 8000`
11. Start Celery workers (2 terminals):
 - `celery -A backend.celery_app worker -Q analysis_queue -c 1`
 - `celery -A backend.celery_app worker -Q notification_queue -c 2`
12. Upload a test resume: `POST /hr/candidates/upload`
13. Wait ~20 seconds — check the candidates table for `resume_embedding` (should be 768 floats)
14. Test semantic search: `GET /hr/candidates/semantic-search?query=Python+backend+developer`
15. Run ngrok and test a voice call: `POST /hr/candidates/{id}/call-now`

Verification Checklist

ollama list shows: llama3.1:8b, mistral:7b, nomic-embed-text

nvidia-smi shows Ollama using GPU VRAM after first model load

`SELECT * FROM pg_extension WHERE extname='vector';` returns a row

After upload: `SELECT array_length(resume_embedding, 1) FROM candidates;` returns 768

Semantic search returns candidates ranked by meaning, not keyword match

HNSW indexes exist: `SELECT indexname FROM pg_indexes WHERE indexname LIKE '%vec%';`

Celery `analysis_queue` worker is running — no GPU conflict during active calls

Twilio webhook URL is ngrok HTTPS — not HTTP (Twilio requires HTTPS)